

Embedded C++

Nathan Warner



**Northern Illinois
University**

Computer Science
Northern Illinois University
United States

Contents

1	General Cpp	2
1.1	Smart pointers	17
1.2	Rvalue references and perfect forwarding	24
2	Threads and Thread safety	35
2.1	Threads and locks	35
2.2	Atomics	46
3	The Embedded Memory Model	60
4	Bit manipulations	64
5	Deducing Types	66
6	Exercises	69

General Cpp

- **Explicit:** In C++, the `explicit` keyword is a specifier used to prevent the compiler from performing implicit type conversions and copy-initialization. It acts as a safety guardrail, forcing developers to explicitly state their intent when converting data types, which helps eliminate a massive category of accidental, hard-to-find bugs.

By default, if a class has a constructor that can be called with a single argument, the C++ compiler treats it as a converting constructor. This means the compiler will silently convert the argument's type into your class type whenever it thinks it needs to

```
0  class Wallet {
1  public:
2      int dollars;
3      // Converting constructor (No explicit keyword)
4      Wallet(int amount) : dollars(amount) {}
5  };
6
7  void printWalletAmount(const Wallet& w) {
8      std::cout << "Wallet has amount: " << w.dollars <<
9      ↪ std::endl;
10 }
11
12 int main() {
13     // Implicit conversion happens here!
14     // The compiler automatically converts the integer '42'
15     ↪ into a Wallet object.
16     printWalletAmount(42);
17
18     // This also works via implicit copy-initialization
19     Wallet myWallet = 100;
20 }
```

If we don't want this implicit conversion, we use `explicit`.

```
0  class Wallet {
1  public:
2      int dollars;
3
4      // Marked as explicit
5      explicit Wallet(int amount) : dollars(amount) {}
6  };
```

Now we must either explicitly call the constructor or explicitly cast into a `Wallet` type.

- **Explicit User-Defined Conversion Operators (C++11):** Starting with C++11, you can also apply `explicit` to conversion operators. This prevents a class from automatically downgrading or converting itself into a primitive type when passed around

```

0  class SmartPointer {
1  public:
2      bool isValid() const { return true; }
3
4      // Marked as explicit conversion to bool
5      explicit operator bool() const {
6          return isValid();
7      }
8  };
9
10 int main() {
11     SmartPointer ptr;
12
13     // Okay, loops and conditions are explicit contexts
14     if (ptr) { ... }
15
16     // Error
17     int x = ptr + 5;
18 }

```

Prior to C++11, `explicit` was only meaningful for single-argument constructors. With the introduction of braced init-lists `()`, constructors with multiple arguments can also participate in implicit conversions, meaning they should often be marked `explicit` too

`explicit` by default unless you have an intentional architectural reason to allow implicit conversions (e.g., a custom `String` class allowing literal `const char*` conversions).

If you separate your class implementation, the `explicit` keyword goes only on the declaration inside the class header file, not on the definition

- **NRVO (Named return value optimization):** NRVO (more commonly abbreviated as NRVO, or Named Return Value Optimization) is a C++ compiler optimization that constructs a named local variable directly in the caller's memory space to avoid copying or moving

Without optimization, a local object is created in the function, copied (or moved) to the return location, and then the local object is destroyed. The compiler builds the named local object directly at the destination address provided by the caller

Zero copies and zero moves happen, even if your object has a complex copy constructor. Unlike unnamed return value optimization (RVO) which became mandatory in C++17, named NRVO remains optional for compilers, though all major compilers (GCC, Clang, MSVC) support it

- **`std::move` use or don't use**

- 1.

```

0  class User {
1  public:
2      explicit User(std::string name)
3          : name_{ /* name or std::move(name)? */ } { ...
4              ↵ }
5  private:
6      std::string name_;
7  };

```

You should use `std::move(name)` here:

- If the caller passes a temporary string (rvalue), it is moved into the parameter, and then moved into the member. Total cost: two cheap moves, zero deep copies.
- If the caller passes an existing string (lvalue), it is copied into the parameter, and then moved into the member. Total cost: one copy, one cheap move.

Pass-by-Value + `std::move`: Passing by value and then moving it into the member variable is the standard modern C++ idiom (frequently called "sink parameters"). It is highly efficient because:

2.

```

0  std::string create_message() {
1      std::string message = "Operation completed";
2      return /* message or std::move(message)? */;
3  }

```

No `std::move`

Because `message` is a local object being returned by value, the compiler can apply named return value optimization (NRVO), constructing it directly in the caller's destination.

If NRVO is not applied, the language permits `message` to be moved automatically. Writing `std::move(message)` explicitly can prevent NRVO, potentially making the program perform a move that could have been eliminated entirely.

3.

```

0  void store(std::vector<std::unique_ptr<int>>& values) {
1      auto number = std::make_unique<int>(42);
2
3      values.push_back(
4          /* number or std::move(number)? */
5      );
6  }

```

It should use

```
0 values.push_back(std::move(number));
```

`std::unique_ptr` cannot be copied because it represents exclusive ownership. `std::move(number)` allows `push_back` to invoke the move constructor, transferring ownership into the vector.

4.

```
0 void process(const std::string& source) {
1     std::string copy{
2         /* source or std::move(source)? */
3     };
4 }
```

No `std::move`

`std::move(source)` would produce a `const std::string&&`. The usual move constructor expects a non-const rvalue reference:

Moving generally modifies the source, so it cannot accept a `const` object. Consequently, `std::move(source)` would still invoke the copy constructor here.

5.

```
0 class Buffer {
1 public:
2     Buffer& operator=(Buffer&& other) {
3         if (this != &other) {
4             release();
5
6             data_ = /* other.data_ or
7                  ↪ std::move(other.data_)? */;
8             size_ = other.size_;
9
10            other.data_ = nullptr;
11            other.size_ = 0;
12        }
13        return *this;
14    }
15
16 private:
17     char* data_ = nullptr;
18     std::size_t size_ = 0;
19
20     void release();
21 };
```

No `std::move`.

Because `data_` is a raw pointer, `std::move(other.data_)` would have no useful effect. Moving and copying a raw pointer both copy its address.

If the member were a `std::unique_ptr<char[]>`, then `std::move` would be necessary:

- **noexcept constructors:** In C++, you should always prioritize marking move constructors as `noexcept`, followed by any other constructor that can structurally guarantee it will never throw an exception

Failing to mark eligible constructors with `noexcept` prevents the compiler from optimizing your code and can cause standard library containers to severely underperform

You should almost always mark your move constructors as `noexcept`

Standard library containers like `std::vector` guarantee strong exception safety. When a vector grows and needs to reallocate its internal memory, it must move the existing elements to the new storage

If a move constructor is not marked `noexcept`, the vector cannot risk using it. If an exception throws mid-move, the original data is altered and corrupted. To stay safe, `std::vector` will fall back to copying your elements, completely defeating the performance benefits of move semantics

- **std::string_view:** `std::string_view` is a lightweight, non-owning view of character data. It usually contains only a pointer and a length. A function accepting it can efficiently read from several sources without allocating a new string:

```
0 void display(std::string_view text);
1
2 display("literal");
3 display(std::string{"temporary"});
4 display(existing_string);
```

The risk is that `std::string_view` does not own or extend the lifetime of the characters. Storing a view after its source is destroyed or modified can leave it dangling:

- **Object lifetime model:** In C++, an object's lifetime is a strict runtime property defined by the abstract machine. It dictates the exact boundaries within which an object is considered "alive" and valid to access. Accessing an object outside its lifetime results in undefined behavior

An object passes through five distinct phases in its complete lifecycle

1. **Storage Allocation:** Memory with the proper size and alignment is obtained.
2. **Initialization (Lifetime Begins):** The constructor completes.
3. **Active Use:** The program reads, modifies, or calls methods on the object.
4. **Destruction (Lifetime Ends):** The destructor starts running.
5. **Storage Deallocation:** The raw memory is released or reused

- **When does lifetime begin and end:** According to the C++ standard, the precise bounds depend on whether the object has a "trivial" or "non-trivial" initialization/destructor:

Object Type	Lifetime Begins When...	Lifetime Ends When...
Trivial Types(e.g., int, float, trivial structs)	Storage with proper size and alignment is obtained.	The storage is reused, released, or overwritten.
Class Types(with non-trivial constructor/destructor)	Initialization is complete (the constructor finishes execution).	The destructor call starts.

An object's lifetime generally begins when suitably aligned storage has been obtained and its initialization is complete. Merely allocating raw bytes does not necessarily create an object in them; a non-implicit-lifetime class may require construction, often through placement new or `std::construct_at`.

For a class object, its lifetime ends when its destructor begins, not when it finishes. An object's lifetime can also end when its storage is released or reused by another object.

- **Construction / destruction order:** Members are initialized in the order they are declared inside the class: first, then second. Their order in the member-initializer list does not control initialization.

More completely, construction proceeds as follows:

- Virtual base classes
- Direct base classes
- Data members in declaration order
- Constructor body

- **Init list vs constructor body assignment:** Members should generally be initialized in the constructor's member-initializer list instead of assigned inside the constructor body?

The initializer list constructs each member directly with its intended value:

```
0  Widget::Widget(int n) : value{n} {}
```

Assignment in the constructor body first initializes the member, often by default construction, and then assigns another value:

```
0  Widget::Widget(int n) {
1      value = n;
2  }
```


That can perform unnecessary work and does not work for members that cannot be assigned after construction. Reference members and const members must be initialized in the initializer list. A member with no accessible default constructor must also be explicitly initialized there. Base-class constructors are selected there as well.

- **Mutable keyword:** The mutable keyword in C++ allows a class data member to be modified even if the containing object is declared as const. It serves as a tool to preserve logical constness, meaning the object appears completely unchanged from the outside, but its internal, non-observable state can still be updated

C++ uses mutable in two distinct scenarios: class data members and lambda expressions

1. **Class data members:** By default, a const member function cannot modify any data members of its class. However, marking a variable as mutable overrides this restriction.

Can be only applied to non-static, non-const, and non-reference data members

```

0  class MathWizard {
1  private:
2      double input;
3      mutable double cachedResult = 0.0;
4      mutable bool isCached = false; // Tracks if cache is
        ↳ valid
5
6  public:
7      MathWizard(double val) : input(val) {}
8
9      // Declared const because it doesn't change the
        ↳ user-facing state
10     double getExpensiveCalculation() const {
11         if (!isCached) {
12             // Modifying mutable variables inside a
                ↳ const function
13             cachedResult = input * 42.0; // Imagine a
                ↳ heavy math formula here
14             isCached = true;
15             std::cout << "[Computing fresh result]\n";
16         } else {
17             std::cout << "[Returning cached result]\n";
18         }
19         return cachedResult;
20     }
21 };
22
23 int main() {
24     const MathWizard wizard(10.0); // Object is
        ↳ completely const
25
26     std::cout << wizard.getExpensiveCalculation() <<
        ↳ "\n"; // Computes
27     std::cout << wizard.getExpensiveCalculation() <<
        ↳ "\n"; // Uses cache
28 }

```

Mutable provides no thread safety. If multiple threads increment hits without synchronization, there is a data race and therefore undefined behavior. Thread safety would require something such as:

2. **Lambda expressions:** By default, variables captured by a lambda by-value are treated as const inside the lambda body. You cannot modify them unless you explicitly mark the lambda as mutable

By-value lambda captures are const by default because the lambda's capture block generates a compiler-defined class, and its operator() method is marked const.

When you write a lambda, the C++ compiler transforms it behind the scenes into a standard struct or class called a closure object. The variables you capture by value become member variables of this generated object.

```

0  int x = 10;
1  auto my_lambda = [x]() { /* code */ };

```

The compiler translates that into a unique class that looks roughly like this:

```

0  // What the compiler creates for you:
1  class AnonymousLambdaClosure {
2  private:
3      int x; // The captured variable becomes a data
           ↪ member
4
5  public:
6      AnonymousLambdaClosure(int ext_x) : x(ext_x) {}
7
8      // CRITICAL: The function call operator is
           ↪ automatically marked const!
9      void operator()() const {
10         // Inside here, 'x' is treated as const because
           ↪ the method is const
11     }
12 };
13
14 AnonymousLambdaClosure my_lambda(x);

```

Because `operator()` is a `const` member function, it is illegal to modify any of the class's internal member variables (like `x`) inside the function body.

In functional programming, a lambda is conceptually treated as a pure, stateless function execution block rather than a stateful object. Making the execution operator `const` aligns lambdas with this philosophy, ensuring that calling a lambda multiple times yields predictable behavior unless you explicitly opt into statefulness using `mutable`.

- **Rule of 3, 5, and 0:** The idea is that if you define any special member function for your class you should probably define all of them.
 - **Rule of Three:** destructor, copy constructor, and copy-assignment operator.
 - **Rule of Five:** adds the move constructor and move-assignment operator.
 - **Rule of Zero:** delegate ownership to RAII types such as `std::vector`, `std::string`, and smart pointers so the compiler-generated special members are sufficient.

These are design guidelines, not requirements to implement every function. After considering them, some operations may appropriately be `= default` or `= delete`.

- **Note about copy assignment:** Copy assignment must safely replace existing allocation without leaking it. It should also handle allocation failure without corrupting the original state and account for self-assignment. Copy-and-swap is one common solution.
- **`std::swap` and `std::copy`:** `std::swap` exchanges the values of two objects, while `std::copy` transfers a range of elements from one location to another. Both are core components of the C++ Standard Template Library (`<utility>` and `<algorithm>` respectively).

`std::swap` takes two objects by reference and swaps their values. In C++11 and later, it typically uses move semantics to avoid heavy copies.

Very fast for primitive types (optimized into registers) and large containers (like `std::vector`, where it only swaps internal pointers, capacity, and size instead of elements)

```
0  int a = 5;
1  int b = 10;
2  std::swap(a, b); // a is now 10, b is now 5
```

`std::copy` copies a range of elements from a source location to a destination location.

It takes three iterators: first and last defining the source range, and `d_first` pointing to the beginning of the destination range.

The destination range must already be allocated and large enough to hold the incoming data (or use back-inserters like `std::back_inserter` for containers).

```
0  std::vector<int> src = {1, 2, 3};
1  std::vector<int> dest(3); // Must be pre-allocated!
2  std::copy(src.begin(), src.end(), dest.begin());
3  // dest is now {1, 2, 3}
```

`std::copy` is the C++ equivalent of `memcpy`, but it is much safer and smarter.

- **Copy and swap (copy assignment):**

```
0  class Buffer {
1      std::size_t size_ = 0;
2      int* data_ = nullptr;
3
4      public:
5      Buffer(std::size_t size)
6          : size_{size}, data_{new int[size]} {}
7
8      Buffer(const Buffer& other)
9          : size_{other.size_}, data_{new int[other.size_]} {
10          std::copy(
11              other.data_,
12              other.data_ + size_,
13              data_
14          );
15      }
16
17      ~Buffer() {
18          delete[] data_;
19      }
```

```

0      friend void swap(Buffer& first, Buffer& second) noexcept
      ↪ {
1          using std::swap;
2          swap(first.size_, second.size_);
3          swap(first.data_, second.data_);
4      }
5
6      Buffer& operator=(Buffer other) {
7          swap(*this, other);
8          return *this;
9      }
10 };

```

The assignment parameter is deliberately passed by value. For `a = b`, `other` is first copy-constructed from `b`. If allocation fails during that copy, the assignment body never starts and `a` remains unchanged. Once the copy succeeds:

1. `a` exchanges resources with `other`.
2. `a` now owns the copied data.
3. `other` owns `a`'s old data.
4. `other` is destroyed and releases the old data.

Self-assignment also works. `a = a` creates an independent copy before anything is changed, swaps it into `a`, and safely destroys the old allocation.

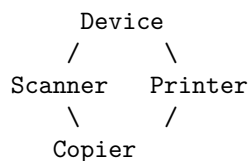
- **Why is marking a move constructor `noexcept` particularly important when objects of that type are stored in a `std::vector`? During vector reallocation, why might the vector copy elements instead of moving them?:** During reallocation, `std::vector` must transfer its elements into new storage. If it moves several elements and a later move throws, the earlier source elements may already have been modified, making rollback difficult.

To preserve the strong exception guarantee, the vector generally copies when:

- The move constructor might throw, and
- The type is copy-constructible.

A `noexcept` move constructor tells the vector that moving cannot fail, allowing it to use the usually cheaper move operation. If the type is move-only, the vector may have no choice but to use a potentially throwing move.

- **Virtual inheritance and the diamond problem:** The diamond problem occurs when a class inherits from two classes that share the same base class.



In ordinary multiple inheritance, `Copier` contains two separate `Device` base-class sub-objects: one inherited through `Scanner` and another through `Printer`

```

0  class Device {
1      public:
2          std::string name;
3  };
4
5  class Scanner : public Device {};
6  class Printer : public Device {};
7
8  class Copier : public Scanner, public Printer {};
9
10 int main() {
11     Copier copier;
12
13     copier.name = "Office copier"; // Error: ambiguous
14 }

```

The compiler cannot determine which `Device::name` you mean:

```

0  copier.Scanner::name = "Scanner side";
1  copier.Printer::name = "Printer side";

```

These are genuinely two different variables because the object conceptually looks like this:

```

Copier
|-- Scanner
|   |-- Device
|       |-- name
|-- Printer
|   |-- Device
|       |-- name

```

This duplication is often undesirable. A real copier is usually intended to be one device that supports scanning and printing, not two independent devices.

Virtual inheritance tells C++ that all relevant inheritance paths should share one base-class subobject.

The intermediate classes must inherit the common base using `virtual`:

```

0  class Scanner : virtual public Device {};
1  class Printer : virtual public Device {};
2
3  class Copier : public Scanner, public Printer {};

```

Now the object conceptually looks like this:

```
Copier
|-- Scanner --|
|-- Printer --|-- shared Device
|-----|      |-- name
```

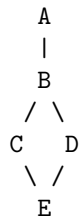
Consequently, accessing name is no longer ambiguous:

Note: Both paths must use virtual inheritance. Virtual inheritance must be specified by every inheritance path that is supposed to share the base

One especially important rule is that the most-derived class constructs the virtual base.

When a Copier is created, it is Copier, not Scanner or Printer, that constructs the shared Device:

Consider



The diamond converges at B, so C and D should inherit virtually from B:

```
0  class A {};
1
2  class B : public A {};
3
4  class C : virtual public B {};
5  class D : virtual public B {};
6
7  class E : public C, public D {};
```

Now an E object contains:

- One shared B subobject
- One A subobject contained inside that shared B
- B is a virtual base of C, D, and ultimately E.
- A is an ordinary base of B.
- E has only one A because it has only one shared B, and that B contains one A.

Also remember that because B is virtual, E, as the most-derived class, is responsible for constructing it:

```

0  class B : public A {
1      public:
2      explicit B(int value) {}
3  };
4
5  class C : virtual public B {
6      public:
7      C() : B(1) {}
8  };
9
10 class D : virtual public B {
11     public:
12     D() : B(2) {}
13 };
14
15 class E : public C, public D {
16     public:
17     E() : B(3) {} // E constructs the shared virtual B
18 };

```

The B(1) and B(2) initializers are ignored while constructing an E; E's B(3) initializes the single shared B.

Both the C and D constructors are still called. Virtual inheritance only changes who constructs the shared B base.

When constructing an E object:

- E first constructs its virtual base B.
 - * Before B's constructor body runs, B constructs its ordinary base A.
- E constructs its first direct base, C.
- E constructs its second direct base, D.
- E constructs its own data members in declaration order.
- The body of E's constructor runs.

Destruction occurs in the exact reverse order:

```

0  Destroy E
1  Destroy D
2  Destroy C
3  Destroy B
4  Destroy A

```

B is constructed only once because it is a virtual base shared by C and D.

- **C++ Casting:**

- **static_cast:** Performs compile-time conversions between related types.
- **dynamic_cast:** Safely converts pointers or references within a class hierarchy at runtime.

Commonly used when downcasting from a base class pointer to a derived class pointer.

Requires the base class to have at least one virtual function (polymorphism). Returns nullptr (for pointers) or throws an exception (for references) if the cast fails.

- **const_cast**: Adds or removes the const or volatile qualifier from a variable.

Commonly used when passing a const variable to a legacy API that accepts a non-const pointer but does not modify the data.

Modifying an object that was originally declared as const after casting away its constness results in undefined behavior.

- **reinterpret_cast**: Reinterprets the underlying bit pattern of an object as a completely different type.

Low-level programming, such as converting a pointer to an integer (uintptr_t) or casting raw byte buffers.

dynamic_cast is the only C++ cast that performs runtime checking.

1.1 Smart pointers

- **Why is `std::unique_ptr<T>` non-copyable?**: Copying would violate exclusive ownership. A unique pointer represents one owning handle, so copying it would create two owners.
- **How should ownership normally be transferred from one `std::unique_ptr` to another?**: Move the unique pointer.
- **What normally happens when a nonempty `std::shared_ptr` is copied?**: The strong reference count increases
- **What is the principal ownership purpose of `std::weak_ptr`?**: Observe shared ownership without extending lifetime
- **What does calling `lock()` on a `std::weak_ptr` produce?**: A shared pointer, empty if expired
- **Which operation makes a `std::unique_ptr` stop owning its object without deleting that object?**: `release()` returns the raw pointer and leaves the unique pointer empty without invoking its deleter
- **What deletion operation does `std::unique_ptr<T[]>` use by default?**: `delete[]`
- **What is wrong with constructing `std::shared_ptr<T> second(first.get())` when `first` already owns the object?**: It can cause the object to be deleted twice

The two shared pointers use separate control blocks, so each may independently delete the same raw pointer

- **Why can a cycle made entirely of `std::shared_ptr` objects leak memory?**: Every object in the cycle can remain strongly owned

Internal strong references can prevent every count in the cycle from reaching zero after external owners disappear.

- **Which change commonly breaks an unintended `std::shared_ptr` ownership cycle?**: Replace one non-owning relationship with `std::weak_ptr`.
- **Cycle problem**: Suppose two objects own each other with `std::shared_ptr`. Neither reference count can reach zero, so neither object is destroyed.

```
0  class Person {
1      public:
2          std::string name;
3          std::shared_ptr<Person> partner;
4
5          explicit Person(std::string name) :
6              ↪ name(std::move(name)) {
7              std::cout << name << " created\n";
8          }
9
10         ~Person() { std::cout << name << " destroyed\n"; }
11     };
```

```

0  int main() {
1      auto alice = std::make_shared<Person>("Alice");
2      auto bob   = std::make_shared<Person>("Bob");
3
4      alice->partner = bob;
5      bob->partner   = alice;
6  }

```

Notice that there are two shared pointers pointing to the same memory, so the reference count for both Person instances is two.

When main() ends:

1. The bob smart-pointer variable is destroyed, bobs reference count decreases to one, bob not destroyed because alice->partner still owns it.
2. The alice smart-pointer variable is destroyed, alice reference count decreases to one, alice not destroyed because bob->partner still owns it.

Therefore neither Bob nor Alice have their destructors called. Each object keeps the other alive, so neither reference count reaches zero.

With weak_ptr, the partner references do not increase the strong reference counts. Therefore, destroying bob reduces Bob's count from one to zero, and destroying alice does the same for Alice. Both Person objects are then correctly destroyed.

- **Unique pointer core api:**

- **Creation:**

```

0  auto p = std::make_unique<MyType>(arg1, arg2);
1  auto array = std::make_unique<int[]>(100);

```

Prefer std::make_unique over writing new directly.

- **Access:**

```

0  p->member;
1  *p;
2  p.get();           // Non-owning raw pointer
3  static_cast<bool>(p);
4  if (p) { }

```

get() does not transfer ownership. The returned pointer must not normally be deleted.

- **Transfer ownership:**

```

0  auto p2 = std::move(p);

```

After the move, p is normally empty. A unique pointer cannot be copied

– **Replace or destroy:**

```
0 p.reset();           // Delete object and become empty
1 p.reset(new MyType); // Delete old object and own new
   ↳ object
2 p = nullptr;         // Delete object and become empty
```

Prefer assigning `std::make_unique` when replacing an object:

```
0 p = std::make_unique<MyType>();
```

– **Release ownership:**

```
0 MyType* raw = p.release();
```

`release()` makes p empty without deleting the object. The caller becomes responsible for it:

– **Swap:**

```
0 p.swap(p2);
1 std::swap(p, p2);
```

– **Custom deleter:**

```
0 struct FileCloser {
1     void operator()(FILE* file) const {
2         if (file)
3             std::fclose(file);
4     }
5 };
6
7 std::unique_ptr<FILE, FileCloser> file(
8     std::fopen("data.txt", "r")
9 );
```

The deleter is part of the `unique_ptr` type:

```
0 std::unique_ptr<FILE, FileCloser>
```

• **Shared pointer core api**

– **Creation:**

```
0 auto p = std::make_shared<MyType>(arg1, arg2);
```

You can copy a shared pointer:

```
0 You can copy a shared pointer:
```

– Access:

```
0 p->member;
1 *p;
2 p.get(); // Non-owning raw pointer
3 if (p) { }
```

– Inspect the strong reference count:

```
0 long count = p.use_count();
```

– Release one owner:

```
0 p.reset();
1 p = nullptr;
```

This decreases the strong reference count. The object is destroyed only when the count reaches zero.

– Replace the managed object:

```
0 p = std::make_shared<MyType>();
1 p.reset(new MyType);
```

– Move ownership:

```
0 auto p2 = std::move(p);
```

Moving does not increase the reference count. Ownership represented by p is transferred to p2.

– Swap:

```
0 p.swap(p2);
1 std::swap(p, p2);
```

– Convert from `unique_ptr`: A `unique_ptr` can transfer ownership into a `shared_ptr`:

```
0 std::unique_ptr<MyType> unique =
1 std::make_unique<MyType>();
2
3 std::shared_ptr<MyType> shared = std::move(unique);
```

The reverse conversion is not generally available because shared ownership cannot safely become exclusive without additional guarantees.

- **Pointer casts:** Use smart-pointer aware casts

```
0 auto derived =
1   std::static_pointer_cast<Derived>(base);
2
3 auto checked =
4   std::dynamic_pointer_cast<Derived>(base);
5
6 auto constant =
7   std::const_pointer_cast<const MyType>(p);
```

These preserve the shared control block.

C++ provides four primary pointer cast functions for `std::shared_ptr` via `<memory>`

- * **`std::static_pointer_cast`:** Casts a pointer up or down an inheritance hierarchy at compile time using `static_cast`. It performs no runtime checks, so you must ensure the conversion is valid
- * **`std::dynamic_pointer_cast`:** Safely downcasts a polymorphic base class pointer to a derived class pointer at runtime using `dynamic_cast`. It returns an empty `std::shared_ptr` if the object is not of the target type.
- * **`std::const_pointer_cast`:** Adds or removes `const` qualifiers from the managed object type using `const_cast`
- * **`std::reinterpret_pointer_cast`:** Reinterprets the underlying pointer type to an unrelated type using `reinterpret_cast`. This is low-level and rarely needed

- **Custom deleter:**

```
0 std::shared_ptr<FILE> file(
1     std::fopen("data.txt", "r"),
2     [](FILE* file) {
3         if (file)
4             std::fclose(file);
5     }
6 );
```

Unlike `unique_ptr`, the deleter is not part of the `shared_ptr` variable's type.

- * **`std::unique_ptr<T, Deleter>`** requires the deleter type and is part of the class type.
- * **`std::shared_ptr<T>`** does not include the deleter in its template parameters; you pass the deleter to the constructor instead

`std::shared_ptr` already uses a heap-allocated control block for reference counts, so storing a type-erased deleter inside that control block adds no extra pointer overhead per smart pointer instance

- **Weak pointer core api:** Use to observe an object managed by `shared_ptr` without keeping it alive.

- **Creation:**

```

0  std::shared_ptr<MyType> shared =
1  std::make_shared<MyType>();
2
3  std::weak_ptr<MyType> weak = shared;

```

- Safely access the object:

```

0  if (auto locked = weak.lock()) {
1      locked->do_something();
2  }

```

lock() returns:

- * A nonempty shared_ptr if the object is still alive.
- * An empty shared_ptr if the object has been destroyed.

The returned shared_ptr keeps the object alive while it is being used.

- Test for expiration:

```

0  if (weak.expired()) {
1      // Object has been destroyed
2  }

```

- Inspect the strong count:

```

0  long count = weak.use_count();

```

- Stop observing:

```

0  weak.reset();

```

- `std::enable_shared_from_this`: Use when an object managed by shared_ptr needs to produce another shared_ptr to itself.

```

0  class Widget : public
    ↪ std::enable_shared_from_this<Widget> {
1  public:
2      std::shared_ptr<Widget> get_pointer() {
3          return shared_from_this();
4      }
5  };
6
7  auto widget = std::make_shared<Widget>();
8  auto another_owner = widget->get_pointer();

```

Do not construct a new shared_ptr from this:

```
o std::shared_ptr<Widget>(this); // Dangerous
```

That creates a separate control block and can cause double deletion. `shared_from_this()` requires the object to already be managed by an appropriate `shared_ptr`. Otherwise, it throws `std::bad_weak_ptr`.

C++ 17 also provides

```
o std::weak_ptr<Widget> weak = weak_from_this();
```


1.2 Rvalue references and perfect forwarding

- **Rvalue references:** Before C++11, passing or returning massive objects (like a `std::vector` with millions of elements) often forced the compiler to deeply copy all the data

With rvalue references, if the compiler knows an object is a temporary (an rvalue), it can trigger a move constructor instead of a copy constructor. Instead of duplicating the heap data, the new object simply "steals" the internal pointers of the temporary object, reducing the operation's computational cost to near-zero

```
0 // A typical move constructor uses an rvalue reference
1 Widget(Widget&& other) noexcept {
2     this->data = other.data; // Steal the pointer
3     other.data = nullptr;   // Clean up the temporary
    ↪ object so its destructor is safe
4 }
```

If you have an lvalue (a named variable) that you are finished with and want to treat like a temporary to trigger a move, you can cast it using `std::move`. `std::move` does not actually move anything; it simply casts an lvalue into an rvalue reference

```
0 std::vector<int> v1 = {1, 2, 3};
1 std::vector<int> v2 = std::move(v1); // v2 steals v1's data.
    ↪ v1 is now empty.
```

This is the most common trap when working with rvalue references: **If an rvalue reference has a name, the expression evaluating it is an lvalue.**

```
0 void process(std::vector<int>&& param) {
1     // 'param' is an rvalue reference type, but the
    ↪ *expression* 'param' is an lvalue!
2
3     std::vector<int> internal1 = param;           // TRAP:
    ↪ Triggers a COPY constructor!
4     std::vector<int> internal2 = std::move(param); //
    ↪ CORRECT: Triggers the MOVE constructor.
5 }
```

- **Perfect forwarding:** When writing generic wrapper functions (like factory functions, `std::make_unique`, or thread constructors), you often need to take arguments and pass them exactly as they are to another function.

To do this perfectly, you must preserve:

- **The value category:** If the user passed an rvalue, the target function should get an rvalue (to trigger a move). If they passed an lvalue, it should get an lvalue.
- **Const modifiers:** If it was `const`, it must stay `const`

Before C++11, achieving this required a massive explosion of function overloads to cover every combination of `const`, non-`const`, lvalue, and rvalue references.

A forwarding reference (originally termed a universal reference by Scott Meyers) looks exactly like an rvalue reference (T&&), but it behaves completely differently. It can bind to lvalues (as a normal reference) and rvalues (as an rvalue reference), preserving constness.

A reference becomes a forwarding reference only when the two following conditions are met:

- **Type deduction is involved:** The type must be deduced by the compiler.
- **The syntax is exactly T&&:** It cannot have modifiers like const T&&

```
0  template<typename T>
1  void wrapper(T&& arg) { // Deduced type + T&& syntax =
    ↪ Forwarding Reference!
2      // ...
3  }
```

The following are NOT universal references.

```
0      void foo(int&& arg); // Not a template. This is strictly
    ↪ an rvalue reference.
1
2      template<typename T>
3      void bar(const T&& arg); // Has 'const'. This is strictly
    ↪ an rvalue reference.
4
5      template<typename T>
6      class Widget {
7          void baz(T&& arg); // 'T' belongs to the class, not
    ↪ the function.
8  }; // No type deduction occurs when
    ↪ calling baz(), so it's an rvalue reference.
```

How can T&& magically bind to an lvalue? The secret lies in a compiler mechanism called reference collapsing.

In C++, you cannot write a reference to a reference directly (e.g., int& && will fail to compile). However, the compiler generates them behind the scenes during template type deduction. To resolve them, it applies a simple rule: An lvalue reference always wins.

- & + & → &
- & + && → &
- && + & → &
- && + && → &&

```

0  template<typename T>
1  void wrapper(T&& arg);
2
3  int x = 10;
4  wrapper(x); // 1. You pass an lvalue (x).
5  // 2. Compiler deduces T as 'int&'.
6  // 3. Substituting T gives: 'int& && arg'
7  // 4. Collapsing rule applies: 'int& arg' (an lvalue
   ↪ reference!)
8
9  wrapper(20); // 1. You pass an rvalue (20).
10 // 2. Compiler deduces T as 'int'.
11 // 3. Substituting T gives: 'int&& arg' (an rvalue
   ↪ reference!)

```

- **std::forward**: As established with rvalue references, once a reference has a name, it evaluates as an lvalue. Inside our wrapper function, the variable `arg` has a name. Therefore, if we just pass `arg` to a target function, it will always be treated as an lvalue, defeating the whole purpose!

Unlike `std::move` (which unconditionally turns an object into an rvalue), `std::forward<T>` conditionalizes its behavior based on how `T` was deduced:

- If `T` was deduced as an lvalue reference (`int&`), `std::forward` casts it back to an lvalue reference.
- If `T` was deduced as a normal type (`int`), `std::forward` casts it back to an rvalue reference.

```

0  #include <iostream>
1  #include <utility>
2
3  void target(int& x) { std::cout << "Lvalue overload\n"; }
4  void target(int&& x) { std::cout << "Rvalue overload\n"; }
5
6  template<typename T>
7  void perfectWrapper(T&& arg) {
8      // std::forward restores the original value category
9      target(std::forward<T>(arg));
10 }
11
12 int main() {
13     int a = 5;
14
15     perfectWrapper(a); // Prints: Lvalue overload
16     perfectWrapper(10); // Prints: Rvalue overload
17 }

```

- **Type vs value category**: In C++, every single expression is characterized by two completely independent properties: its type and its value category
 - **Type** answers the question: "What kind of data is this, and what operations can I perform on it?"

- **Value category:** Value Category answers the questions: "Does this data have a permanent address in memory (identity), and can I safely steal its resources (movability)?"

Value categories are a property of expressions, not variables or objects. An expression is any piece of code that evaluates to a value (like `x`, `5 + 3`, or `foo()`)

Modern C++ (C++11 and later) classifies expressions based on two core criteria:

- **Identity:** Does the program have a way to determine the exact memory address of the object
- **Movability:** Can the object's resources be safely hijacked/moved because its lifetime is ending?
- **The 3 Primary Value Categories:** Every expression belongs to exactly one of these three primary buckets:
 - **lvalue (Left-hand value):** Has identity but cannot be safely moved. These are typically names of variables or things that persist in memory.
 - **prvalue (Pure rvalue):** Has no identity but can be moved. These are pure mathematical values or temporary objects that do not exist yet beyond the evaluation.
 - **xvalue (eXpiring value):** Has identity and can be moved. It points to a real place in memory, but the developer has explicitly given permission to strip its resources because it is about to expire.
- **Mixed categories:** The C++ standard groups these primary categories into two broader terms
 - **glvalue (Generalized lvalue):** Anything with identity (lvalue + xvalue).
 - **rvalue:** Anything that can be moved (prvalue + xvalue)

The single most confusing concept in C++ is that a variable's type is independent of the value category of the expression containing its name

```

0  void process(int&& arg) {
1      // The TYPE of 'arg' is an rvalue reference (int&&)
2      // The VALUE CATEGORY of the expression 'arg' is an
        ↪ lvalue!
3      int y = arg;
4  }
```

- **auto deduction vs CTAD:** While both features automate type resolution in C++, they operate at completely different stages of object creation. `auto` deduces the type of a variable from its initializer expression, whereas CTAD (Class Template Argument Deduction) deduces a class template's missing parameters from its constructor arguments.
 - **Auto:** Deduces the complete type of a variable, uses **template function type deduction** rules

It strips top-level `const`, `volatile`, and references by default, unless you explicitly decorate it (e.g., `const auto&`).
 - **CTAD:** Deduces missing template arguments for a class. Uses **constructor signatures** and implicit / explicit deduction guides

Before C++17, you always had to explicitly state template arguments when instantiating a class, or rely on helper factory functions like `std::make_pair`.

With CTAD, you can omit the template brackets `<...>` entirely. The compiler looks at the arguments passed to the constructor and figures out what the template parameters should be.

```
0 // Pre-C++17
1 std::pair<int, double> p1(1, 2.3);
2 auto p2 = std::make_pair(1, 2.3); // Workaround using
  ↪ auto
3
4 // Post-C++17 (CTAD)
5 std::pair p3(1, 2.3);           // Automatically
  ↪ deduces std::pair<int, double>
```

You will frequently see `auto` and CTAD used in the exact same line of code. They do not conflict; they handle two separate responsibilities:

```
0 auto v = std::vector{1, 2, 3};
```

- **CTAD steps in first:** It sees `std::vector{1, 2, 3}`, looks at the integer arguments, and resolves the class type to `std::vector<int>`.
- **auto steps in second:** It looks at the resolved right-hand side (`std::vector<int>`) and assigns that exact type to the variable `v`.
- **auto vs decltype:** `auto` and `decltype` both perform type inference at compile time, but they use different deduction rules and serve distinct purposes

The core difference is that `auto` deduces a type based on an initialization value (similar to template argument deduction), whereas `decltype` yields the exact declared type of an expression or variable without evaluating it

- **Auto:** Declaring a variable whose type matches its initializer.
- **decltype(expr):** Inspecting the exact type of a variable or an expression.

`decltype` also preserves references and `const`. The important thing to note is that `decltype` does not require an initializer, and does not evaluate the expression.

- **Questions:**
 1. **What kinds of expressions can bind to `int&&`? How does that differ from what can bind to `const int&`?:** Both pure rvalues (prvalues) and lvalues cast to an rvalue reference (with `std::move`) can bind to `int&&`.

Both prvalues and lvalues can bind to `int&`.

```
0 void f(int&& x) { ... }
1
2 int x = 10;
3 f(10); // prvalue binds
4 f(std::move(x)) // xvalue binds
```

```

0 void g(const int& x) { ... }
1 g(10); // prvalue binds
2 g(x); // lvalue binds

```

2. Does `std::move(x)` move anything by itself? What does it do to the expression `x`? `std::move(x)` does not move data; it casts `x` to an rvalue reference, making the resulting expression an xvalue. A move may happen later if that expression is used to call a move constructor or move assignment operator.
3. Inside this function, is the expression `s` an lvalue or an rvalue? Why?:

```

0 void f(std::string&& s) {
1     use(s);
2 }

```

`s` has rvalue-reference type, but the expression `s` is an lvalue because it names an object. To pass it on as an rvalue, you would write `std::move(s)`.

The rvalue-reference type matters mainly at the function boundary: it determines what arguments may be passed to `f`.

```

0 void f(std::string&& s);
1
2 std::string name = "Ada";
3
4 f(name); // Error: lvalue cannot bind to
   ↪ string&&
5 f(std::move(name)); // OK: xvalue
6 f(std::string{"Ada"}); // OK: prvalue

```

Inside `f`, merely writing `s` treats it as an lvalue. To take advantage of the fact that `f` received an object that may be moved from, explicitly use `std::move(s)`:

4. What are the deduced type `T` and the resulting parameter type `T&&` in each call and explain how reference collapsing affects each result:

```

0 template<class T>
1 void f(T&& x);
2
3 std::string name = "Ada";
4
5 f(name);
6 f(std::string{"Ada"});

```

For the lvalue

```

0 f(name);

```

T is deduced as `std::string&`, referencing collapsing yields $\& + \&\& = \&$. Thus, the function argument is type `std::string&`.

For the temporary:

```
0 f(std::string{"Ada"});
```

T is deduced as `std::string`. $T\&\&$ therefore becomes `std::string\&\&`. This makes $T\&\&$ a forwarding reference because T is deduced from the argument.

5. **Why should this wrapper use `std::forward<T>(value)` instead of `std::move(value)`? Assume `consume` has overloads for lvalues and rvalues:**

```
0 template<class T>
1 void wrapper(T\&\& value) {
2     consume(/* value */);
3 }
```

- If the original argument was an lvalue, T is $U\&$, so forwarding produces an lvalue.
- If the original argument was an rvalue, T is U , so forwarding produces an xvalue.
- `std::move(value)` would produce an xvalue in both cases and could incorrectly move from an original lvalue.

Note: Consider

```
0 template<typename T>
1 void f(T\&\& arg) { ... }
2
3 template<typename T>
4 void g(T arg) { ... }
5
6 int x{5};
7 f(x); g(x);
```

In the call to g , the type is deduced as `int`, whereas in f , the type is deduced as `int\&`. The fundamental difference in behavior exists to prevent silent, unintentional copying of objects while enabling perfect forwarding.

In C++, template type deduction rules are explicitly designed around the signature of the function parameter. The compiler treats T arg and $T\&\&$ arg with two entirely different sets of rules when it encounters an lvalue.

- **Implementation problem:** Implement `forward_to_consume` so it calls the appropriate overload:

```

0  void consume(const std::string&) {
1      std::cout << "lvalue\n";
2  }
3
4  void consume(std::string&&) {
5      std::cout << "rvalue\n";
6  }
7
8  template<typename T>
9  void forward_to_consume(T&& value) {
10     std::forward<T>(value);
11 }
12
13 int main() {
14     std::string name = "Ada";
15
16     forward_to_consume(name);                // Must print:
17     ↪ lvalue
18     forward_to_consume(std::string{"Grace"}); // Must print:
19     ↪ rvalue
20 }

```

- **Dependent name qualifier:** A dependent name is a name that depends on a template parameter.

Because the compiler doesn't know what type will eventually replace the template parameter during compilation, it cannot look up what a dependent name actually refers to until the template is instantiated

```

0  template <typename T>
1  void process(T obj) {
2      int x = 10;        // 'int' and 'x' are Non-Dependent
2      ↪ Names. They mean the same thing regardless of T.
3      T::value_type y;   // 'value_type' is a Dependent Name.
3      ↪ What it means depends entirely on what T is.
4  }

```

Dependent names generally fall into two categories, both of which require special keywords to help the compiler parse the code before instantiation.

1. **Dependant types (requires typename)** Because the compiler defaults to assuming nested names are member variables or functions, you must explicitly prefix it with `typename`.

```

0  template <typename T>
1  void print_size() {
2      // Compiler error without 'typename' because it
2      ↪ assumes 'size_type' is a static variable
3      typename T::size_type size = 0;
4  }

```


2. **Dependant templates (requires template):** A member accessed through a dependent type might itself be a template

```
0  template<typename T>
1  void call(T& object) {
2      object.template convert<int>();
3  }
```

The template keyword tells the compiler that convert is a member template and that <int> contains template arguments.

Without it:

```
0  object.convert<int>();
```

the compiler might interpret < as the less-than operator because it does not yet know what members T has.

```
0  struct Converter {
1      template<typename U>
2      U convert() {
3          return U{};
4      }
5  };
6
7  template<typename T>
8  void use(T& object)
9  {
10     int result = object.convert<int>();
11 }
```

Use typename when a dependent qualified name represents a type. Use template when a member accessed through a dependent expression is a template.

- **Dependent base classes:** A base class can depend on a template parameter:

```
0  template<typename T>
1  struct Base {
2      void print() {}
3  };
4
5  template<typename T>
6  struct Derived : Base<T> {
7      void run()
8      {
9          print(); // May fail
10     }
11 }
```

Base<T> is a dependent base class. During the initial parsing of Derived, the compiler does not search dependent base classes for unqualified names.

You can qualify the call:

```
0  template<typename T>
1  struct Derived : Base<T> {
2      void run() {
3          this->print();
4      }
5  };
```

Other valid approaches include:

```
0  Base<T>::print();
```

or:

```
0  using Base<T>::print;
```

The this-> expression is dependent because the type of this depends on T. Therefore, lookup for print can be delayed until instantiation.

- **Two-phase name lookup:** Templates are generally checked in two phases.
 1. **Phase 1: Template definition:** The compiler parses the template and resolves names that do not depend on template parameters.

```
0  void helper();
1
2  template<typename T>
3  void function(T value)
4  {
5      helper();          // Non-dependent
6      value.work();      // Dependent
7  }
```

helper() can be looked up immediately. The validity of value.work() depends on T.

2. **Phase 2: Template instantiation:** When the template is used with a concrete argument, dependent names and expressions are resolved:

```
0  struct Worker {
1      void work() {}
2  };
3
4  function(Worker{}); // value.work() becomes valid
5  function(42);      // Error: int has no work()
```

This delayed checking is one reason template errors often appear at the point where the template is instantiated.

- **Uses of T&&:** T&& has two closely related uses:
 1. An ordinary rvalue reference binds to rvalues and enables move semantics, allowing resources such as a vector's heap allocation to be transferred instead of copied.
 2. In a deduced template context, such as `template<class T> void f(T&& x)`, it may be a forwarding reference. It can accept both lvalues and rvalues, and `std::forward<T>(x)` preserves the argument's original value category.

Threads and Thread safety

2.1 Threads and locks

- Creating a thread

```
0  #include <iostream>
1  #include <thread>
2
3  void worker(int id) {
4      std::cout << "Worker " << id << '\n';
5  }
6
7  int main() {
8      std::thread t(worker, 1);
9
10     t.join();
11 }
```

The operating system schedules the new thread independently. You should not assume which thread runs first or how their instructions will be interleaved.

- **Join:** A `std::thread` is normally handled with `join()`.

```
0  std::thread t(do_work);
1  t.join();
```

`join()` blocks the calling thread until `t` finishes. A thread object must not be destroyed while it is still joinable. Doing so calls `std::terminate()`:

A thread must be joined to make the calling thread wait until the target thread finishes its work, prevent the main program from exiting too early, and safely clean up system resources

You can check its state:

```
0  if (t.joinable())
1      t.join();
```

- **Detached threads:**

```
0  t.detach();
```

A detached thread continues independently. The `std::thread` object no longer controls it.

Detached threads are dangerous because the program must ensure that everything the thread accesses remains alive:

```
0 void dangerous() {
1     int value = 42;
2
3     std::thread([&value] {
4         use(value); // value might already be destroyed
5     }).detach();
6 }
```

- **jthread**: C++20 introduced `std::jthread`. Its destructor automatically joins the thread.

```
0 #include <thread>
1
2 void worker() {
3     // Work
4 }
5
6 int main() {
7     std::jthread t(worker);
8 } // Automatically joined
```

- **Passing arguments**: Arguments passed to `std::thread` are normally copied or moved into internal storage.

```
0 void process(int value);
1
2 int x = 10;
3 std::thread t(process, x); // x is copied
```

To deliberately pass by reference, use `std::ref`:

```
0 #include <functional>
1
2 void increment(int& value) {
3     ++value;
4 }
5
6 int x = 10;
7 std::thread t(increment, std::ref(x));
8 t.join();
```

- **std::mutex**: A mutex provides mutual exclusion: only one thread can own it at a time.

```

0  #include <mutex>
1
2  int counter = 0;
3  std::mutex counter_mutex;
4
5  void increment()
6  {
7      for (int i = 0; i < 100'000; ++i) {
8          counter_mutex.lock();
9          ++counter;
10         counter_mutex.unlock();
11     }
12 }

```

The protected region is called a critical section. Do not normally call `lock()` and `unlock()` manually. If an exception or early return occurs before `unlock()`, the mutex remains locked

- **RAII locking:** C++ locking should normally use RAII: lock in a constructor, unlock in a destructor.

Use `std::lock_guard` for a simple scope-based lock:

```

0  void increment() {
1      for (int i = 0; i < 100'000; ++i) {
2          std::lock_guard<std::mutex> lock(counter_mutex);
3          ++counter;
4      }
5  }

```

The mutex is automatically unlocked at the end of the scope.

You can shorten the critical section:

```

0  void function(){
1      perform_unrelated_work();
2
3      {
4          std::lock_guard<std::mutex> lock(counter_mutex);
5          modify_shared_state();
6      }
7
8      perform_more_unrelated_work();
9  }

```

Keeping critical sections short reduces contention.

When you initialize a `std::lock_guard`, it calls `.lock()` on the underlying mutex (usually `std::mutex`). If another thread holds the lock, the operating system puts the requesting thread to sleep (context switch).

- **`std::unique_lock`**: `std::unique_lock` is more flexible:

```
0  std::unique_lock<std::mutex> lock(counter_mutex);
1
2  modify_shared_state();
3
4  lock.unlock();
5  perform_expensive_work();
6
7  lock.lock();
8  modify_shared_state_again();
```

It supports:

- Deferred locking
- Manual unlock/relock
- Ownership transfer
- Condition variables
- Timed mutex operations

It has slightly more state and overhead than `std::lock_guard`.

- **`std::scoped_lock`** is particularly useful when acquiring several mutexes:

```
0  std::scoped_lock lock(account_a.mutex, account_b.mutex);
```

It uses a deadlock-avoidance algorithm rather than naively locking them in the listed sequence.

- **`try_lock()` and timed mutexes**: `try_lock()` attempts to acquire a mutex without waiting:

```
0  if (mutex.try_lock()) {
1      // Mutex acquired
2      use_shared_state();
3      mutex.unlock();
4  } else {
5      // Do something else
6  }
```

The primary difference is that `std::mutex::lock()` is a blocking operation, while `std::mutex::try_lock()` is a non-blocking operation

RAII can still be used:

```

0  std::unique_lock lock(mutex, std::try_to_lock);
1
2  if (lock.owns_lock()) {
3      use_shared_state();
4  }

```

- **Condition variables:** In C++, `std::condition_variable` is a synchronization primitive that allows one thread to suspend execution (sleep) until another thread modifies a shared variable and signals the condition variable. This eliminates the need for inefficient busy-waiting loops

To use a condition variable, you need three pieces working together

1. **`std::condition_variable`:** The signaling mechanism.
2. **`std::mutex`:** Protects the shared state.
3. **`std::unique_lock<std::mutex>`:** Passed to the `wait()` function. A `std::lock_guard` cannot be used for waiting because the condition variable must frequently unlock and relock the mutex while waiting

```

0  #include <condition_variable>
1  #include <mutex>
2  #include <queue>
3
4  std::queue<int> queue;
5  std::mutex mutex;
6  std::condition_variable available;
7  bool finished = false;

```

```

0  // Producer
1  void produce(int value) {
2      {
3          std::lock_guard lock(mutex);
4          queue.push(value);
5      }
6
7      available.notify_one();
8  }

```



```

0  // Consumer
1
2  void consume() {
3      for (;;) {
4          std::unique_lock lock(mutex);
5
6          available.wait(lock, [] {
7              return !queue.empty() && finished;
8          });
9
10         if (queue.empty() && finished) {
11             break;
12         }
13
14         int value = queue.front();
15         queue.pop();
16
17         lock.unlock();
18         process(value);
19     }
20 }

```

The predicate must be checked because:

- Wakeups can be spurious.
- Another consumer may take the item first.
- A notification means the state may have changed, not that the condition is definitely true.

Note: A spurious wakeup occurs when a thread wakes up from a waiting state (sleeping/blocked on a condition variable or monitor) without receiving any explicit signal, notification, or interrupt, and before the condition it is waiting for has actually been met.

The actual condition is the protected shared state, such as `!queue.empty()`. The condition variable is only the notification mechanism.

- **std::shared_lock:** `std::shared_lock` is a general-purpose, RAII-compliant shared lock ownership wrapper introduced in C++14. It is primarily designed for reader-writer scenarios, allowing multiple "reader" threads to concurrently access a shared resource while ensuring that "writer" threads get exclusive access
- **Reader-writer locking:** `std::shared_mutex` permits:
 - Multiple simultaneous readers
 - One exclusive writer

`std::shared_mutex` permits:

- Multiple simultaneous readers
- One exclusive writer
- **Multiple Readers:** Multiple threads can hold a `std::shared_lock` on the same mutex simultaneously

- **Exclusive Block:** If any thread holds an exclusive lock (e.g., via `std::unique_lock` or `std::lock_guard`), all `std::shared_lock` requests will block until the exclusive lock is released
- **RAII Management:** It automatically calls `lock_shared()` on construction and `unlock_shared()` upon destruction (when it goes out of scope)

```

0  #include <shared_mutex>
1  #include <unordered_map>
2
3  class Dictionary {
4      public:
5          int get(int key) const {
6              std::shared_lock lock(mutex_);
7              return values_.at(key);
8          }
9
10         void set(int key, int value) {
11             std::unique_lock lock(mutex_);
12             values_[key] = value;
13         }
14
15         private:
16             mutable std::shared_mutex mutex_;
17             std::unordered_map<int, int> values_;
18     };

```

This helps primarily when reads are frequent, writes are uncommon, and the protected operations are substantial enough to justify the added complexity. An ordinary mutex may otherwise perform better.

- **One-time initialization:** Use `std::call_once` when initialization must occur exactly once:

```

0  #include <mutex>
1
2  std::once_flag initialization_flag;
3
4  void initialize()
5  {
6      std::call_once(initialization_flag, [] {
7          initialize_library();
8      });
9  }

```

Function-local static initialization is also thread-safe since C++11:

```

0  Resource& resource() {
1      static Resource instance;
2      return instance;
3  }

```

- **Semaphore:** A semaphore represents a number of available permits:

```

0  #include <semaphore>
1
2  std::counting_semaphore<4> slots(4);
3
4  void worker() {
5      slots.acquire();
6      use_limited_resource();
7      slots.release();
8  }

```

Unlike a mutex, a semaphore does not represent ownership of a critical section. It is useful for limiting concurrency or representing available resources.

A semaphore (whether a counting semaphore or a binary semaphore) does not represent ownership because it is a signaling mechanism, not a locking mechanism.

Unlike a mutex (mutual exclusion lock), which is explicitly designed to manage ownership of a critical section, a semaphore focuses strictly on managing a count of available resources.

- **The Rule:** In a mutex, only the exact thread that locked it can unlock it.
- **The Semaphore Difference:** Any thread can call `signal()` (or `post/V`) on a semaphore, even if it never called `wait()` (or `pend/P`) to acquire it. Because there is no restriction tying a specific thread to the resource, no concept of "ownership" exists.

- **Latch:** A `std::latch` is a one-use countdown:

```

0  #include <latch>
1  #include <thread>
2  #include <vector>
3
4  std::latch completed(4);
5
6  void worker() {
7      perform_work();
8      completed.count_down();
9  }
10
11 // Start four workers...
12 completed.wait();

```

`std::latch` is a C++20 thread synchronization primitive that pauses one or more threads until a downward counter reaches zero

- **Single-Use:** Once the internal counter hits zero, the latch is done and cannot be reset or reused.
- **No Increment:** You cannot increase the counter after creation; you only count it down.

`std::latch::wait` blocks the calling thread until the internal counter of the `std::latch` reaches zero.

```

0  #include <latches>
1  #include <thread>
2  #include <vector>
3
4  // Initialize latch with count for 3 workers
5  std::latch work_latch(3);
6
7  for (int i = 0; i < 3; ++i) {
8      std::thread([&work_latch]() {
9          // Do work...
10         work_latch.count_down(); // Signal completion
11     }).detach();
12 }
13
14 work_latch.wait(); // Main thread blocks here until all 3
    ↪ workers finish

```

- **std::barrier:** std::barrier is a C++20 thread-coordination mechanism that blocks a group of participating threads until a specific number of arrivals are reached, after which it can be reused for multiple phases.

Unlike std::latch, which is single-use and destroys its counter value at zero, std::barrier automatically resets and can be used for repetitive coordination cycles

When all threads arrive at the synchronization point, the barrier executes a completion step (callable object) on one of the participating threads before unblocking the rest

```

0  #include <barrier>
1  int main() {
2      // 1. Define a completion function to run at the end of
   ↳ each phase
3      auto on_completion = []() noexcept {
4          ...
5      };
6
7      std::barrier sync_point(2, on_completion);
8
9      auto worker = [&](std::string name) {
10         ...
11
12         // Block until all 2 threads reach this point
13         sync_point.arrive_and_wait();
14
15         ...
16
17         // Reuse the same barrier for the next phase
18         sync_point.arrive_and_wait();
19
20         ...
21     };
22
23     std::vector<std::thread> threads;
24     threads.emplace_back(worker, "Thread A");
25     threads.emplace_back(worker, "Thread B");
26
27     for (auto& t : threads) {
28         t.join();
29     }
30
31     return 0;
32 }

```

- **std::async:** std::async is a function template in C++ (introduced in C++11) that runs a callable object-like a function, lambda, or functor-asynchronously or lazily, returning a std::future that will eventually hold the result. It abstracts away manual thread management, automatically handling execution and allowing you to retrieve return values or handle exceptions across thread boundaries safely
- **std::promise:** A std::promise represents a commitment made by a worker thread to provide a result at some point. The thread fulfills this contract by calling .set_value().
- **std::future:** A std::future represents a placeholder for a value that does not exist yet, but will exist in the future.

When you call .get(), you are looking into the "future." If the future hasn't arrived yet, you sit and wait for it.

- **std::promise with std::future:** In C++, std::promise and std::future are a pair of class templates introduced in C++11 to pass values, exceptions, or signals between threads without explicitly managing mutexes or condition variables. They establish a one-way, single-use communication channel where one thread acts as the producer and another acts as the consumer

```

0  void calculate(std::promise<int> result)
1  {
2      try {
3          result.set_value(42);
4      } catch (...) {
5          result.set_exception(std::current_exception());
6      }
7  }
8
9  int main()
10 {
11     std::promise<int> promise;
12     std::future<int> future = promise.get_future();
13
14     std::thread t(calculate, std::move(promise));
15
16     int result = future.get();
17     t.join();
18 }

```

Promises are useful when the producer and consumer need a direct one-time communication channel.

2.2 Atomics

- **std::atomic<T>**: std::atomic<T> provides thread-safe access to a shared value without requiring an explicit mutex for each operation.

Its main purpose is preventing a data race when multiple threads read or modify the same variable.

Without an atomic variable, this program has undefined behavior

```
0  #include <thread>
1
2  int counter = 0;
3
4  void increment()
5  {
6      for (int i = 0; i < 100'000; ++i) {
7          ++counter;
8      }
9  }
```

++counter is a read-modify-write operation. Two threads can interfere with each other. Instead, we can use std::atomic

```
0  #include <atomic>
1  #include <iostream>
2  #include <thread>
3
4  std::atomic<int> counter{0};
5
6  void increment()
7  {
8      for (int i = 0; i < 100'000; ++i) {
9          ++counter;
10     }
11 }
12
13 int main()
14 {
15     std::thread first(increment);
16     std::thread second(increment);
17
18     first.join();
19     second.join();
20
21     std::cout << counter.load() << '\n'; // 200000
22 }
```

Every increment is now indivisible from the perspective of other threads.

- **Operations:**

```

0  std::atomic<int> value{10};
1
2  value.store(20);           // Atomic write
3  int x = value.load();     // Atomic read
4
5  value.fetch_add(5);        // Add 5; returns the old value
6  value.fetch_sub(2);        // Subtract 2; returns the old
   ↪ value
7
8  value++;                  // Atomic increment
9  value--;                  // Atomic decrement

```

- **Cannot be copied:** Atomic objects generally cannot be copied. You copy the contained value explicitly
- **Atomic flags:** A common use is communicating a simple condition between threads:

```

0  #include <atomic>
1  #include <thread>
2
3  std::atomic<bool> stop_requested{false};
4
5  void worker()
6  {
7      while (!stop_requested.load()) {
8          // Perform work
9      }
10 }
11
12 int main()
13 {
14     std::thread thread(worker);
15
16     // Later:
17     stop_requested.store(true);
18
19     thread.join();
20 }

```

Atomicity ensures that accessing `stop_requested` does not create a data race. If `stop_requested` were a normal `bool`, the worker thread would read it while the main thread writes it. One thread writes the object while another thread reads it, producing a data race. In C++, a data race results in undefined behavior.

Because ordinary C++ code has no defined data races, the compiler may assume another thread does not modify `stop_requested`. It could effectively transform this:

```

0  while (!stop_requested) {
1  }

```


into something resembling:

```
0  bool cached = stop_requested;
1
2  while (!cached) {
3  }
```

The worker might therefore loop forever even after the main thread assigns true.

Each CPU core has registers, caches, and store buffers. A normal write does not provide the synchronization required by the C++ memory model to make another thread observe the change correctly.

Modern hardware has cache-coherence mechanisms, but those mechanisms alone do not make racy C++ code valid. The compiler and processor are both allowed to reorder ordinary operations when no synchronization is present.

On many systems, reading or writing a single byte-sized bool happens as one hardware operation. That only means the reader probably will not observe a partially written value.

Thread safety requires more than preventing a torn read. It also requires:

- Defined concurrent access
- Appropriate visibility between threads
- Restrictions on compiler and processor reordering

A normal bool provides none of these guarantees.

- **Torn read:** A torn read is a concurrency bug that occurs when a thread reads a value from memory while another thread is simultaneously writing to it, resulting in the reader seeing a partially updated, corrupted value
- **Cache coherence:** Cache coherence is the practice of making sure that all copies of shared data stored in different local caches remain identical in a multi-core or multi-processor system
- **Compare-and-exchange:** Compare-and-exchange is central to lock-free algorithms. It means if the atomic currently contains the expected value, replace it with the desired value. Otherwise, update the expected variable with the actual value.

```
0  std::atomic<int> value{10};
1
2  int expected = 10;
3
4  bool changed = value.compare_exchange_strong(expected, 20);
```

- **strong vs weak compare exchange:** `compare_exchange_weak` and `compare_exchange_strong` perform a conditional atomic update called compare-and-exchange, often abbreviated CAS.

Conceptually, CAS performs this indivisible operation:

```
0  if (atomic_value == expected) {
1      atomic_value = desired;
2      return true;
3  } else {
4      expected = atomic_value;
5      return false;
6  }
```

No other thread can interrupt the comparison and replacement as two separate steps.

Consider

```
0  int current = value.load();
1  value.store(current + 1);
```

Two threads could perform this sequence:

Step	Thread A	Thread B	Atomic value
1	Loads 10	10	
2	Loads 10	10	
3	Stores 11	11	
4	Stores 11	11	

One increment is lost. CAS allows each thread to store the new value only if the old value has not changed since it was loaded

The fundamental difference is that `compare_exchange_weak` may report failure even when the atomic object equals expected. This is called a **spurious failure**

A **spurious failure** in a compare-and-exchange (or Compare-and-Swap / CAS) operation occurs when the function returns false (indicating failure) even though the current value in memory exactly matches the expected value. Instead of executing the swap, the operation acts as if the values did not match

On many RISC architectures (like ARM, PowerPC, or MIPS), atomic operations are implemented using a pair of instructions: Load-Link (LL) and Store-Conditional (SC)

The LL instruction tracks the memory address. The SC instruction only succeeds if no changes or specific hardware events occurred at that address since the LL instruction.

Context switches, system interrupts, or even basic CPU cache eviction events can cause the CPU to "lose the reservation" on that memory address. As a result, the SC instruction fails naturally, inducing a spurious failure even if no other thread changed the underlying value

Strong fails only when the comparison fails, it is suitable for a one-time attempt.

Weak may fail because

- The atomic value did not equal expected; or
- The atomic value did equal expected, but the implementation produced a spurious failure.

Using `compare_exchange_strong` in a retry loop is usually considered a performance anti-pattern because `compare_exchange_weak` is more efficient on many CPU architectures.

While `compare_exchange_strong` guarantees it will only fail if the value actually changed, this guarantee comes with a hardware cost that is unnecessary inside a loop.

So, weak is normally used in a loop.

- **Memory ordering:** Atomic operations have two related responsibilities:

1. Making an individual operation indivisible.
2. Controlling how memory operations become visible between threads.

Memory ordering defines how compilers and CPU hardware are allowed to reorder read and write operations across multiple threads

- **Single-threaded code:** Compilers and CPUs change instruction order to make code run faster. This reordering is invisible because the final result stays the same.
 - **Multi-threaded code:** Other threads can spot these reordered operations. Without rules, one thread might see updates in a completely different order than another thread wrote them, breaking synchronization
- **Store-Buffer Forwarding (Store-Load Reordering):** Imagine two threads running on two different CPU cores, sharing two variables (X and Y), both starting at 0.

Thread 1 (Core 1)	Thread 2 (Core 2)
1. Write 1 to X	3. Write 1 to Y
2. Read Y	4. Read X

Logically, it should be impossible for both threads to read 0. At least one thread has to write its 1 before the other thread reads.

Yet, without memory barriers, both reads can return 0.

Writing data to the main RAM (or even the L1 cache) is slow. To speed things up, each CPU core has a tiny, ultra-fast temporary holding zone called a Store Buffer.

1. Core 1 executes Write 1 to X. Instead of waiting for X to travel to cache, Core 1 drops the write into its local Store Buffer and immediately moves on.
2. Core 1 executes Read Y. It checks its own Store Buffer (nothing for Y there) and reads Y from the cache. Because Core 2 hasn't flushed its own buffer yet, Y is still 0.

3. Core 2 does the exact same thing simultaneously. It drops Write 1 to Y into its own Store Buffer and immediately executes Read X, which returns 0.
4. Finally, both cores flush their Store Buffers out to the shared cache.

To stop the CPU from doing this, you must use a Memory Barrier (like C++'s `std::memory_order_seq_cst`). This instruction forces the CPU core to completely empty its local Store Buffer and commit the data to shared cache before it is allowed to perform the subsequent Read.

- **Memory barrier:** A memory barrier (also called a memory fence) is a specific CPU instruction that forces the processor and the compiler to maintain a strict order of memory operations. It acts as a wall that prevents reads and writes from crossing past it.

Barrier Type	What It Does	Common Use Case
LoadLoad	Ensures older reads finish before newer reads can start.	Preventing a thread from reading stale data.
StoreStore	Ensures older writes are visible to cache before newer writes happen.	Guaranteeing a payload is written before turning on a "Ready" flag.
LoadStore	Ensures older reads finish before newer writes can happen.	Preventing early overwrites of data.
StoreLoad	Ensures older writes are completely flushed before newer reads start.	The heaviest barrier. Required to prevent the store-buffer shortcut described earlier.

- **More on memory ordering:** Memory ordering controls how operations surrounding an atomic operation may be observed by other threads.

Every `std::atomic` operation is atomic regardless of its memory order. The ordering argument determines whether that operation also establishes ordering and visibility for other memory accesses.

Consider

```

0  int data = 0;
1  std::atomic<bool> ready{false};
2
3  void producer() {
4      data = 42;
5      ready.store(true);
6  }
7
8  void consumer() {
9      while (!ready.load()) { }
10
11     std::cout << data << '\n';
12 }

```

The intent is:

- Producer writes data.
- Producer sets ready.
- Consumer observes ready.
- Consumer reads data.

For this to work, the atomic flag must do more than safely transfer a Boolean value. It must also ensure that the write to data becomes visible before the consumer reads it. Memory ordering defines that relationship.

- **Program order is not necessarily observation order:** Within one thread, the source code says:

```
0  data = 42;  
1  ready = true;
```

But several layers may reorder or delay operations:

- The compiler may rearrange instructions.
- The CPU may execute or retire memory operations out of order.
- Stores may temporarily remain in a store buffer.
- Different cores may observe stores at different times.

These transformations are allowed when they do not change the observable behavior of a correctly synchronized C++ program.

Atomic memory orders constrain these transformations.

Importantly, the C++ memory model is the authority. Hardware cache coherence alone does not make unsynchronized C++ code valid.

- **Sequenced-before:** ordering inside one thread.
 - **Synchronizes-with:** ordering established between threads through synchronization operations.
 - **Happens-before:** the combined ordering that makes earlier effects visible and prevents data races.
- **Global order:** A global order in thread memory models is a single, total sequence of operations that every thread in a program agrees on and observes in the exact same way

If Thread A performs an operation before Thread B in this global sequence, every other thread in the system will witness the events in that exact same chronological order

CPUs and compilers cannot reorder operations across different threads in a way that violates this shared global sequence

It mimics a simple single-threaded program where instructions happen one after another, making concurrent code easy to reason about

- **Even more on memory ordering:** `std::memory_order` controls two separate properties of an atomic operation:
 1. The atomic operation itself: whether other threads can observe it partially.
 2. The ordering of other memory operations around it: whether ordinary reads and writes become visible across threads.

Every `std::atomic` operation is atomic regardless of its memory order. The difference is synchronization.

Order	Atomicity	Acquires earlier writes	Publishes earlier writes	Global order
‘relaxed’	Yes	No	No	No
‘consume’	Yes	Dependency-based historically	No	No
‘acquire’	Yes	Yes	No	No
‘release’	Yes	No	Yes	No
‘acq_rel’	Yes	Yes	Yes	No
‘seq_cst’	Yes	Yes	Yes	Yes, among sequentially consistent operations

Note that modern spelling is

```
0  std::memory_order::relaxed
1  std::memory_order::acquire
2  std::memory_order::release
3  std::memory_order::acq_rel
4  std::memory_order::seq_cst
```

The older spellings, such as `std::memory_order_relaxed`, are also commonly seen.

- **Release and acquire:**

```
0  int data = 0;
1  std::atomic<bool> ready{false};
```

Thread 1:

```
0  data = 42;
1  ready.store(true, std::memory_order::release);
```

Thread 2:

```
0 while (!ready.load(std::memory_order::acquire)) {
1 }
2
3 std::cout << data; // Guaranteed to print 42
```

1. data = 42 sequenced-before
2. release store of true synchronizes-with
3. acquire load that reads true sequenced-before
4. reading data

Therefore, writing data happens-before reading data.

The critical condition is that the acquire load must observe the value from the release store. Merely using release and acquire somewhere in the program is insufficient.

if the load reads false, the consumer keeps waiting. When it eventually reads the producer's true, the release/acquire synchronization is established.

“Sequenced-before” describes ordering within one thread. Because statement A appears before statement B in the producer thread, A is sequenced-before B:

The release store effectively publishes everything that the producer did before it. Therefore, the write to data is part of what is being published.

- **memory_order_relaxed**: Relaxed ordering guarantees atomicity and a per-object modification order, but it creates no synchronization for other memory.

This is appropriate when you only need an accurate counter. Concurrent increments cannot overwrite each other.

However, relaxed ordering cannot normally publish surrounding data:

```
0 int data = 0;
1 std::atomic<bool> ready{false};
2
3 // Thread 1
4 data = 42;
5 ready.store(true, std::memory_order::relaxed);
6
7 // Thread 2
8 if (ready.load(std::memory_order::relaxed)) {
9     std::cout << data; // Data race: undefined behavior
10 }
```

The atomic ready variable is safe, but nothing connects the ordinary write to data with the ordinary read from data.

- **memory_order_release**: Release ordering is used on stores and read-modify-write operations.

```
0 ready.store(true, std::memory_order::release);
```

Its purpose is publication: operations sequenced before the release can become visible to a thread that performs a matching acquire operation.

Valid operations are

```
0 atomic.store(value, std::memory_order::release);
1 atomic.exchange(value, std::memory_order::release);
2 atomic.fetch_add(value, std::memory_order::release);
```

This is invalid:

```
0 atomic.load(std::memory_order::release); // Invalid
```

A load does not publish a value, so release ordering does not make sense for a pure load.

- **memory_order_acquire**: Acquire ordering is used on loads and read-modify-write operations.

```
0 bool value = ready.load(std::memory_order::acquire);
```

If this operation reads from a corresponding release operation, everything published before that release becomes visible after the acquire.

An acquire load does not publish preceding operations to other threads. It receives information published by another thread.

A pure store does not read a value, so it cannot acquire anything.

- **memory_order_acq_rel**: Acquire-release combines both directions:
 - Acquire the writes published by another thread.
 - Release the current thread's preceding writes.

It is intended for atomic read-modify-write operations:

```
0 atomic.fetch_add(1, std::memory_order::acq_rel);
1 atomic.exchange(value, std::memory_order::acq_rel);
2 atomic.compare_exchange_weak(
3     expected,
4     desired,
5     std::memory_order::acq_rel
6 );
```


A read-modify-write operation both reads the old value and publishes a new value, so it can meaningfully perform both functions.

```
0  std::atomic<int> state{0};
1
2  int old = state.fetch_add(1, std::memory_order::acq_rel);
```

The operation can:

- Acquire information associated with the value it reads.
- Release preceding work through the new value it writes.

This is invalid for pure loads and stores

```
0  atomic.load(std::memory_order::acq_rel); // Invalid
1  atomic.store(1, std::memory_order::acq_rel); // Invalid
```

Use acquire for the load and release for the store.

- **memory_order_seq_cst**: Sequential consistency provides acquire/release behavior plus a single total order over all sequentially consistent atomic operations.

```
0  atomic.store(1); // seq_cst
1  int x = atomic.load(); // seq_cst
```

The program behaves as though all `seq_cst` operations were placed into one global sequence that every thread agrees upon. The standard requires a single total order for these operations

Consider

```
0  std::atomic<int> x{0};
1  std::atomic<int> y{0};
2
3  int r1;
4  int r2;
```

```
0  // Thread 1
1  x.store(1, std::memory_order::seq_cst);
2  r1 = y.load(std::memory_order::seq_cst);
3
4  // Thread 2
5  y.store(1, std::memory_order::seq_cst);
6  r2 = x.load(std::memory_order::seq_cst);
```

With exclusively sequentially consistent operations, this result is forbidden:

```
0  r1 == 0 && r2 == 0
```

To obtain both zeros, each load would need to appear before the other thread's store, producing an impossible cycle in the global sequentially consistent order.

- **Release/acquire synchronization is conditional:** An acquire load synchronizes with a release store only when the load reads the value written by that store.

So, with

```
0  x.store(1, std::memory_order::release);
1  r1 = y.load(std::memory_order::acquire);
2
3  // Thread 2
4  y.store(1, std::memory_order::release);
5  r2 = x.load(std::memory_order::acquire);
6
7  // r1 == r2 == 0 possible
```

For thread 1's load

```
0  r1 = y.load(std::memory_order::acquire);
```

- If it reads 1, it read from Thread 2's release store, so synchronization occurs.
- If it reads the original 0, it did not read from Thread 2's release store, so no synchronization occurs.

The same applies to Thread 2's load of x. Therefore, if both loads read the initial values, neither acquire load read from the other thread's release store

Note: The standard explicitly restricts stores to relaxed, release, or sequentially consistent ordering, and loads to relaxed, acquire, or sequentially consistent ordering

- **Compare-exchange success and failure orders:** Compare-exchange often takes two orders:

```
0  value.compare_exchange_weak(
1      expected,
2      desired,
3      std::memory_order_acq_rel, // Success
4      std::memory_order_acquire  // Failure
5  );
```

On success, it reads the old value and writes the new one, so acq_rel is possible.

On failure, it only reads the actual value into expected. Because no write occurred, the failure order cannot contain release semantics. Therefore, release and `acq_rel` are invalid failure orders.

A common pattern is

```
0  while (!head.compare_exchange_weak(  
1      expected,  
2      desired,  
3      std::memory_order_release,  
4      std::memory_order_relaxed  
5  )) {  
6      // expected was updated  
7  }
```

The successful operation publishes data. Failure merely retrieves the current value and does not need synchronization.

- **Atomics do not make an entire algorithm atomic:** Suppose two atomic variables must remain consistent with each other:

```
0  std::atomic<int> account_a{100};  
1  std::atomic<int> account_b{100};  
2  
3  void transfer() {  
4      account_a.fetch_sub(50);  
5      account_b.fetch_add(50);  
6  }
```

Each operation is atomic, but the complete transfer is not. Another thread could observe:

```
0  account_a == 50  
1  account_b == 100
```

If both values must change as one transaction, use a mutex:

```
0  #include <mutex>  
1  
2  int account_a = 100;  
3  int account_b = 100;  
4  std::mutex accounts_mutex;  
5  
6  void transfer() {  
7      std::lock_guard lock(accounts_mutex);  
8  
9      account_a -= 50;  
10     account_b += 50;  
11 }
```

Atomics protect individual atomic operations. They do not automatically protect larger invariants involving several operations or objects.

- **Lock-free does not always mean faster:** An atomic type may be implemented directly using hardware instructions, or the implementation may internally use locks.

You can inspect this for a type

```
0  std::atomic<long long> value{0};
1
2  if (value.is_lock_free()) {
3      // This atomic object is implemented without locks.
4  }
```

For a compile-time check when available:

```
0  static_assert(std::atomic<int>::is_always_lock_free);
```

Simple types such as bool, pointers, and properly aligned machine-sized integers are commonly lock-free, but portable code should not assume this unless the implementation guarantees it.

- **Atomic pointers:** Pointers can also be atomic:

```
0  int values[10];
1
2  std::atomic<int*> pointer{values};
3
4  pointer.fetch_add(1); // Advances by one int
5  int* current = pointer.load();
```

Atomic pointers are important in lock-free data structures, although safe memory reclamation makes those structures substantially more complicated than they initially appear.

- **C++20 waiting:** C++20 allows a thread to wait for an atomic value to change without manually spinning continuously:

```
0  std::atomic<bool> ready{false};
1
2  void consumer() {
3      ready.wait(false);
4
5      // ready is no longer false
6  }
7
8  void producer() {
9      ready.store(true);
10     ready.notify_one();
11 }
```

The Embedded Memory Model

- **Intro:** Embedded C++ uses the same language-level memory model as desktop C++, but embedded systems usually adopt a different allocation policy

Allocate everything at startup or compile time, then avoid dynamic allocation during normal operation.

Heap use is discouraged in embedded software because dynamic memory allocation introduces unpredictable behavior, memory fragmentation, and fatal crash risks that violate safety-critical requirement

Functions like `malloc()` or `new` take variable amounts of time to search for free memory blocks, which ruins real-time deadlines where timing must be exact

Allocating and freeing differently sized blocks over time breaks RAM into small, unusable gaps. A system can report plenty of free memory overall yet still fail a new allocation request

Small embedded devices have strict RAM limits. Memory leaks or unexpected allocation spikes cause the heap to run out, leading to hard faults or system resets

The stack and the heap grow toward each other in RAM; when they collide, the system crashes

- **Typical MCU memory layout:** A bare-metal firmware image commonly uses memory like this:

```
Flash
|-- Vector table
|-- Machine code (.text)
|-- Constants (.rodata)
|-- Initial values for global variables

SRAM
|-- Initialized globals/statics (.data)
|-- Zero-initialized globals/statics (.bss)
|-- Heap          usually grows upward
|-- Unused RAM
|-- Stack         usually grows downward
```

Code and constant data normally reside in flash, while mutable data, stacks, and any heap reside in SRAM.

- **Static-storage objects:** Objects can be allocated globally or as static objects. Their storage is reserved before the program begins normal execution. There is no runtime allocation and no fragmentation.

The downside is that these objects exist for the firmware's entire lifetime and may complicate initialization or testing if overused.

- **Automatic stack allocation:** Short-lived objects are normally local variables:

```

0 void process_message() {
1     std::array<std::byte, 64> message{};
2     Parser parser{message};
3
4     parser.process();
5 }

```

The compiler adjusts the stack pointer when the function begins and restores it when the function returns. There is no allocator involved, so allocation is fast and deterministic.

However, MCU stacks are limited. Avoid putting very large buffers on the stack:

Large buffers are generally better placed in static storage or a deliberately managed memory region.

- **Fixed-capacity containers:** Instead of containers that grow dynamically, embedded code often uses containers whose maximum capacity is known at compile time:

```

0 template<typename T, std::size_t Capacity>
1 class StaticVector {
2     std::array<T, Capacity> storage_;
3     std::size_t size_ = 0;
4
5     public:
6     bool push_back(const T& value)
7     {
8         if (size_ == Capacity)
9             return false;
10
11         storage_[size_++] = value;
12         return true;
13     }
14 };

```

This provides vector-like behavior without heap allocation.

- **Fixed-size memory pools:** When objects must be created and destroyed dynamically, a fixed pool is often preferable to a general-purpose heap:

```

0 struct Message {
1     std::uint32_t id;
2     std::array<std::byte, 32> payload;
3 };
4
5 std::array<Message, 16> message_pool;

```

A pool maintains a collection of equally sized slots. Allocation selects a free slot, and deallocation returns it.

- Fixed maximum memory usage
- No external fragmentation
- Usually constant or bounded allocation time
- Straightforward failure behavior
- Easier testing of exhaustion

- **Arena allocation:** An arena owns a fixed region and dispenses memory sequentially:

```
0  class Arena {
1      std::byte* current_;
2      std::byte* end_;
3
4      public:
5      Arena(void* memory, std::size_t size)
6          : current_{static_cast<std::byte*>(memory)},
7            end_{current_ + size} {}
8
9      void* allocate(std::size_t bytes);
10 };
```

Individual objects usually are not freed. The complete arena is reset when all objects allocated from it are no longer needed.

- **New on bare metal:** The compiler still understands

```
0  Widget* object = new Widget{};
```

Conceptually, this performs two operations:

- Obtain raw storage through operator new.
- Construct Widget in that storage.

```
0  void* memory = operator new(sizeof(Widget));
1  Widget* object = ::new (memory) Widget{};
```

Whether ordinary new works depends on the runtime and linker configuration. In many bare-metal toolchains, the allocator ultimately obtains heap space through a low-level function such as `_sbrk`, which you or the vendor runtime must provide.

You can also disable or forbid heap use by:

- Not implementing the required heap-growth routine
- Overriding global operator new
- Treating allocation calls as link errors

- Enforcing a project rule against dynamic containers
- Using linker symbols to reserve zero or limited heap space

`::new` is a C++ expression that forces the compiler to call the global operator `new` function instead of any custom, class-specific operator `new` override

- **Placement new:** Placement new constructs an object in storage you already own:

```
0  #include <new>
1  #include <stddef>
2
3  alignas(Widget) std::byte storage[sizeof(Widget)];
4
5  Widget* widget = new (storage) Widget{};
6
7  widget->~Widget();
```

Placement new does not allocate memory. It only begins the object's lifetime in the supplied storage.

Alignment and object lifetime must be handled correctly, so higher-level wrappers such as `std::optional`, `std::variant`, or a well-tested pool are generally preferable.

Bit manipulations

- Reverse bit pattern of a 32 bit integer (Iterative):

```
0  std::uint32_t rev(std::uint32_t n) {
1      std::uint32_t res{0};
2      for (size_t i{}; i < 32; ++i) {
3          res = res << 1 | (n & 1);
4          n >>= 1;
5      }
6
7      return res;
8  }
```

- Reverse bit pattern of a 32 bit integer (fast):

```
0  unsigned int reverse_bits_fast(unsigned int n) {
1      // Swap adjacent single bits (01010101 and 10101010)
2      n = ((n & 0x55555555) << 1) | ((n & 0xAAAAAAAA) >> 1);
3      // Swap adjacent 2-bit pairs (00110011 and 11001100)
4      n = ((n & 0x33333333) << 2) | ((n & 0xCCCCCCCC) >> 2);
5      // Swap adjacent 4-bit nibbles (00001111 and 11110000)
6      n = ((n & 0x0F0F0F0F) << 4) | ((n & 0xF0F0F0F0) >> 4);
7      // Swap adjacent 8-bit bytes
8      n = ((n & 0x00FF00FF) << 8) | ((n & 0xFF00FF00) >> 8);
9      // Swap the 16-bit halves
10     n = (n << 16) | (n >> 16);
11     return n;
12 }
```

- Check if power of two

```
0  bool power_of_two(std::uint32_t value) {
1      if (value == 0) return false;
2      return value & (value-1) == 0;
3  }
```

- Clear lowest set bit

```
0  value & (value - 1)
```

- Count number of set bits

```

0  unsigned count_set_bits(std::uint32_t value) {
1      unsigned count{};
2      while (value != 0) {
3          value &= value-1;
4          ++count;
5      }
6      return count;
7  }

```

- Get msb

```

0  std::uint32_t get_msb(std::uint32_t value) {
1      value = value >> 1;
2      value = value >> 2;
3      value = value >> 4;
4      value = value >> 8;
5      value |= value >> 16;
6
7      return value ^= (value >> 1);
8  }

```

- Every value in a vector appears exactly twice except one, which appears once. Find that single value in $O(n)$ time and $O(1)$ extra space:

```

0  int find_single(const std::vector<int>& values);

```

The operator is bitwise XOR ($\wedge$). Two equal values cancel because $x \wedge x == 0$, and XOR with zero leaves a value unchanged. XOR is also associative and commutative, so the pairs need not be adjacent:

```

0  int find_single(const std::vector<int>& values) {
1      int result = 0;
2      for (int ell : values) {
3          result ^= ell;
4      }
5
6      return result;
7  }

```

Deducing Types

- **Rule sets:** C++ has three distinct sets of type deduction rules
 1. **Template type deduction:** Divided into three cases based on the parameter type
 - (a) ParamType is a reference or pointer (but not a universal/forwarding reference).
 - (b) ParamType is a universal/forwarding reference (T&&).
 - (c) ParamType is passed by value (drops references, top-level const, and volatile)
 2. **Auto:** It generally mirrors Template Type Deduction (the variable's type acts like ParamType and the initializer expression acts like the argument).

Braced initializer lists deduce to a `std::initializer_list` under `auto`, which fails under standard template argument deduction

3. **decltype / decltype(auto):** Used to inspect the exact, unmodified type of a variable or expression.

Unlike `auto` and templates, `decltype` does not strip references or top-level cv-qualifiers; it gives you the exact declaration type or reference category (lvalue vs rvalue) of the expression

- **decltype(auto):** `decltype(auto)` is a type specifier introduced in C++14 that tells the compiler to deduce a type using `decltype` rules rather than standard `auto` template deduction rules.
- **Template type deduction:** Consider

```
0  template<typename T>
1  void f(ParamType param);
2
3  f(expr);
```

There are three cases for `ParamType`, each of which effect `T`'s deduction differently

1. **ParamType** is a pointer or reference type. In this case, if the `expr` is a reference, the reference part is ignored. Then pattern-match `expr`'s type against `ParamType` to determine `T`.

```
0  template<typename T>
1  void f(T& param);
2
3  int x = 27;
4  const int cx = x;
5  const int& rx = x;
```

Calling `f` with `x`, `cx`, and `rx` deduces `T` as `int`, `const int`, `const int`.

2. **ParamType** is a universal reference. If `expr` is an lvalue, both `T` and `ParamType` are deduced to be lvalue references. If `expr` is an rvalue, the “normal” (i.e., Case 1) rules apply.

Universal references are described heavily in a section above and will likely be described in more detail later.

3. **ParamType** is neither a pointer nor a reference. As before, if `expr`'s type is a reference, ignore the reference part. If, after ignoring `expr`'s reference-ness, `expr` is `const`, ignore that, too. If it's volatile, also ignore that.

```
0  template<typename T>
1  void f(T param);
2
3  int x = 27;
4  const int cx = x;
5  const int& rx = x;
```

Calling `f` with `x`, `cx`, and `rx` deduces `T` as `int` in every call.

Note that even though `cx` and `rx` represent `const` values, `param` isn't `const`. That makes sense. `param` is an object that's completely independent of `cx` and `rx`—a copy of `cx` or `rx`. The fact that `cx` and `rx` can't be modified says nothing about whether `param` can be. That's why `expr`'s constness (and volatileness, if any) is ignored when deducing a type for `param`: just because `expr` can't be modified doesn't mean that a copy of it can't be.

- **Array references:**

```
0  int arr[10];
1  int (&r)[10] = arr;
```

Here, `r` is a reference to an array of 10 integers. With this, we can have access to the array in a function without being decayed to a pointer

```
0  void f(int (&arr_ref)[10]) {
1      // 40
2      std::cout << sizeof(arr_ref) << "\n";
3  }
```

Now, in

```
0  template<typename T>
1  void f(T& p);
2
3  f(arr);
```

`T` is deduced as `int[10]`, and `p` is `int (&)[10]`;

Interestingly, the ability to declare references to arrays enables creation of a template that deduces the number of elements that an array contains

```
0  template<typename T, std::size_t N> constexpr std::size_t  
    ↪ array_size(T (&)[N]) noexcept { return N; }
```

Exercises

- Fixed capacity ring buffer:

```
0  class RingBuffer {
1      std::size_t capacity_{};
2      std::size_t size_{};
3
4      // Position of the next read
5      std::size_t read_{};
6
7      // Position of the next write
8      std::size_t write_{};
9
10     // Internal storage
11     std::vector<int> buf_{};
12 public:
13     explicit RingBuffer(std::size_t capacity) :
14         ↪ capacity_(capacity), buf_(capacity) {};
15
16     bool push(int);
17     std::optional<int> pop();
18     std::size_t size() const noexcept;
19     bool full() const noexcept;
20     void print() const;
21 };
```

```
0  bool RingBuffer::push(int value) {
1      if (full()) return false;
2
3      buf_[write_] = value;
4      write_ = (write_ + 1) % capacity_;
5      ++size_;
6
7      return true;
8  }
```

```
0  std::optional<int> RingBuffer::pop() {
1      if (size() == 0) return std::nullopt;
2
3      int head = buf_[read_];
4      read_ = (read_ + 1) % capacity_;
5      --size_;
6
7      return head;
8  }
```

```

0  std::size_t RingBuffer::size() const noexcept {
1      return size_;
2  }
3
4  bool RingBuffer::full() const noexcept {
5      return size_ == capacity_;
6  }

```

```

0  void RingBuffer::print() const {
1      std::size_t idx = read_;
2
3      for (size_t i{0}; i < size_; ++i) {
4          if (i != 0) std::cout << ", ";
5          std::cout << buf_[idx];
6          idx = (idx + 1) % capacity_;
7      }
8
9      std::cout << "\n";
10 }

```

- **Merge intervals:** Implement a function that merges overlapping closed intervals:

```

0  struct Interval {
1      int start;
2      int end;
3  };
4
5  std::vector<Interval> merge_intervals(
6  const std::vector<Interval>& intervals
7  );

```

Each input interval satisfies $\text{start} \leq \text{end}$. The input may be unsorted. Return the merged intervals sorted by start, without modifying the input.

```

0  Input:  [5, 7], [1, 3], [3, 4], [10, 12]
1  Output: [1, 4], [5, 7], [10, 12]

```

```

0  std::vector<Interval> merge_intervals(const
    ↪ std::vector<Interval>& intervals) {
1      if (intervals.empty()) return {};
2
3      std::vector<Interval> ret;
4      std::vector<Interval> cpy = intervals;
5
6      if (cpy.size() == 1) return cpy;
7
8      std::sort(cpy.begin(), cpy.end(), [](const Interval& a,
    ↪ const Interval& b) {
9          return a.start < b.start;
10     });
11
12     std::size_t left = 0;
13     while (left < cpy.size()) {
14         Interval merged = cpy[left];
15
16         std::size_t last_merged = left;
17         for (std::size_t i{left+1}; i < cpy.size(); ++i) {
18             if (cpy[i].start <= merged.end) {
19                 merged.end = std::max(merged.end,
    ↪ cpy[i].end);
20                 last_merged = i;
21                 continue;
22             }
23
24             break;
25         }
26
27         if (last_merged == left) ++left;
28         else left = last_merged+1;
29
30         ret.push_back(merged);
31     }
32
33     return ret;
34 }

```